

Part E: Daily Bookings Pipeline in Dagster

The job runs every night at 02:00 WIB for the previous day. It takes the bookings created that day from MySQL, runs each one through the LLM categorizer, and loads the result into ClickHouse for reporting.

```
MySQL bookings -> raw_bookings -> enriched_bookings -> clickhouse_bookings -> reports
```

(a) Assets and daily partitions

Each step is an asset, partitioned by day with `DailyPartitionsDefinition` in the Asia/Jakarta timezone, so "yesterday" means the Jakarta calendar day and not the UTC one. The schedule materialises the latest finished partition, and a backfill is simply a range of partitions.

- **raw_bookings** reads the rows whose `created_at` falls inside the partition window and saves them as a snapshot in object storage under `dt=YYYY-MM-DD`. The later steps read this snapshot instead of MySQL, so a rerun works on exactly the same rows even if the source table has changed since.
- **enriched_bookings** reads the snapshot and adds `category`, `enrichment_status`, the error message if there was one, and the prompt version used.
- **clickhouse_bookings** loads the enriched rows into the matching ClickHouse partition.

Resources are the MySQL read replica, object storage, the LLM categorizer (with a concurrency limit, retries and a result cache) and ClickHouse.

A sketch of the code (not meant to run as-is):

```
daily = DailyPartitionsDefinition(start_date="2026-01-01", timezone="Asia/Jakarta")

@asset(
    partitions_def=daily,
    retry_policy=RetryPolicy(max_retries=3, delay=60),
    check_specs=[
        AssetCheckSpec("row_count_vs_baseline", asset="raw_bookings", blocking=True),
        AssetCheckSpec("critical_fields_not_null", asset="raw_bookings", blocking=True),
        AssetCheckSpec("booking_contract", asset="raw_bookings", blocking=True),
        AssetCheckSpec("amounts_by_source", asset="raw_bookings", blocking=True),
    ],
)
def raw_bookings(context, mysql: MySQLResource, lake: LakeResource):
    window = context.partition_time_window
    df = mysql.query_df(
        "SELECT * FROM bookings WHERE created_at >= %s AND created_at < %s",
        params=(window.start, window.end),
    )
```

```

lake.overwrite(f"raw_bookings/dt={context.partition_key}/", df)

history = lake.partition_stats("raw_bookings", same_weekday_weeks=4,
before=context.partition_key)
return MaterializeResult(
    metadata={"row_count": len(df), "baseline_row_count": history.median_row_count},
    check_results=[
        check_row_count(df, history),
        check_null_rates(df, history),
        check_contract(df),
        check_amounts_by_source(df, history),
    ],
)

@asset(
    partitions_def=daily,
    deps=[raw_bookings],
    check_specs=[AssetCheckSpec("enrichment_failure_rate", asset="enriched_bookings",
blocking=True)],
)
def enriched_bookings(context, lake: LakeResource, categorizer: LLMCategorizerResource):
    df = lake.read(f"raw_bookings/dt={context.partition_key}/")

    # retries each row, caches by (booking_id, input hash, prompt version),
    # and returns failed rows with category=None instead of dropping them
    enriched = categorizer.categorize_all(df, concurrency=20, max_retries=3)
    lake.overwrite(f"enriched_bookings/dt={context.partition_key}/", enriched)

    failed = int((enriched["enrichment_status"] == "failed").sum())
    failure_rate = failed / max(len(enriched), 1)
    return MaterializeResult(
        metadata={"rows": len(enriched), "failed_rows": failed, "failure_rate": failure_rate},
        check_results=[AssetCheckResult(
            check_name="enrichment_failure_rate",
            passed=failure_rate <= 0.02,
            # up to 10%: warn and keep going; above 10%: error, which blocks the load
            severity=AssetCheckSeverity.WARN if failure_rate <= 0.10 else AssetCheckSeverity.ERROR,
            metadata={"failure_rate": failure_rate},
        )],
    )

@asset(partitions_def=daily, deps=[enriched_bookings])
def clickhouse_bookings(context, lake: LakeResource, clickhouse: ClickHouseResource):
    day = context.partition_key # e.g. "2026-09-14"
    df = lake.read(f"enriched_bookings/dt={day}/")

```

```

staging = f"analytics.bookings_enriched_stage_{day.replace('-', '')}"

clickhouse.command(f"CREATE TABLE IF NOT EXISTS {staging} AS analytics.bookings_enriched")
clickhouse.command(f"TRUNCATE TABLE {staging}")
clickhouse.insert_df(staging, df.assign(partition_date=day))
clickhouse.command(f"ALTER TABLE analytics.bookings_enriched REPLACE PARTITION '{day}' FROM
{staging}")

clickhouse.command(f"DROP TABLE {staging}")

rows, amount = clickhouse.query_one(
    "SELECT count(), sum(total_amount) FROM analytics.bookings_enriched WHERE partition_date =
{day:Date}",
    parameters={"day": day},
)
if rows != len(df) or amount != df["total_amount"].sum():
    raise Failure(f"ClickHouse partition {day} does not match the snapshot")

daily_bookings_job = define_asset_job(
    "daily_bookings", selection=[raw_bookings, enriched_bookings, clickhouse_bookings],
    partitions_def=daily
)
daily_bookings_schedule = build_schedule_from_partitioned_job(daily_bookings_job, hour_of_day=2)

```

ClickHouse table:

```

CREATE TABLE analytics.bookings_enriched
(
    partition_date      Date,
    booking_id          UInt64,
    property_id         UInt64,
    property_type       LowCardinality(String),
    checkin_date        Date,
    checkout_date       Date,
    status              LowCardinality(String),
    total_amount        Decimal(15, 2),
    currency            LowCardinality(FixedString(3)),
    source              LowCardinality(String),
    created_at          DateTime('Asia/Jakarta'),
    category            LowCardinality(Nullable(String)),
    enrichment_status   LowCardinality(String),
    prompt_version      LowCardinality(String),
    loaded_at          DateTime DEFAULT now()
)
ENGINE = MergeTree

```

```
PARTITION BY partition_date
ORDER BY (partition_date, source, property_type, booking_id);
```

(b) Idempotency

Nothing in this pipeline appends data. Every step replaces the output for its own partition:

- The snapshot and enriched files live under a path keyed by the date, and a rerun overwrites them.
- In ClickHouse the day is loaded into a staging table first, and then swapped in with `ALTER TABLE ... REPLACE PARTITION ... FROM staging`. The swap is atomic, so rerunning a day leaves exactly one copy of it, and nobody querying the report ever sees half a day.
- The time window always comes from the partition key, never from `now()`, so running 14 September today or next month reads the same range.
- LLM results are cached by booking id, a hash of the fields sent to the model, and the prompt version. A rerun doesn't pay for the LLM again and gets the same categories back. Changing the prompt version is the deliberate way to recategorise.

One trade-off: partitioning ClickHouse by day means about 365 partitions a year. That's fine for a few years of data. For longer retention I'd merge older days into monthly partitions.

(c) The LLM fails on 5% of the rows

The failed rows are not dropped, and they don't fail the run. The categorizer retries each row a few times with backoff. If a row still fails, it continues with category set to NULL and `enrichment_status = 'failed'`, and it is also written to a small dead-letter table with the error. The other 95% load as usual, and the failed 5% are still in ClickHouse with their amounts, so GMV and booking counts in the report stay correct. Those rows simply show up as "Uncategorized" until they're fixed.

How loud this gets depends on the failure rate:

- **Up to 2%:** normal.
- **Between 2% and 10%:** a warning and an alert, but the load goes ahead. A 5% failure rate lands here.
- **Above 10%:** the load is blocked. At that point it's most likely a provider outage, and publishing a day where most categories are empty would do more harm than showing the day as late.

Fixing it just means running the partition again, either by hand or with a small daily job that retries the last few days. Because of the cache, only the rows that failed call the LLM again, and the partition swap makes the retry safe to repeat.

(d) Stopping bad upstream data

The checks on `raw_bookings` are blocking asset checks. When one fails with ERROR severity, Dagster doesn't run the enrichment or the load for that day. ClickHouse keeps what it had, the report shows the day as delayed rather than wrong, and we get an alert with the numbers from the failing check.

What gets checked:

- **Row count** against the median of the same weekday over the previous four weeks. Less than half or more than double fails the run; a swing of 30% only warns. This covers the case where the row count suddenly halves.
- **Null rate** of the columns the report relies on: `total_amount`, `currency`, `checkin_date`, `checkout_date`, `property_type`, `status` and `source`. The run fails if a column is 100% null, or if its null rate is more than 5 points above its own recent level. That catches a column that was renamed or dropped upstream.
- **Basic contract**: unique ids, `status` and `currency` within the allowed values, checkout after check-in, amount above zero, and `created_at` inside the window.
- **Amounts per source**: the average nightly rate per property type for each source, compared with its last 28 days. This is the check that would have caught the `partner_x` problem from C3 on its first day. A source value we've never seen before raises a warning, so someone looks at it before it becomes part of the numbers.
- **Freshness**: the newest `created_at` in MySQL should be close to the end of the window. If replication is behind, we retry later instead of loading a half-empty day.
- **After the load**, the row count and `sum(total_amount)` in the ClickHouse partition have to match the snapshot.

Real spikes do happen, around Lebaran or during a big promo. For those, someone reruns the day with an explicit override and a short reason in the run config. The override is stored with the run, which is better than quietly loosening the thresholds for everyone.

Notes

- `bookings` has no `updated_at` column, so partitioning by `created_at` freezes a booking's status at the moment it was extracted. A booking created yesterday as pending and completed today would stay pending in the report. The cheap fix is to rerun the last 30 partitions every night, which the LLM cache keeps inexpensive. The proper fix is change data capture from the MySQL binlog into a `ReplacingMergeTree` table.
- Amounts should be converted to IDR during the transform step, with the exchange rate stored on each row, so the report never adds IDR and USD together.